Parcours d'arbres en OCaml

Dans tout le début du TP, on utilisera le type suivant :

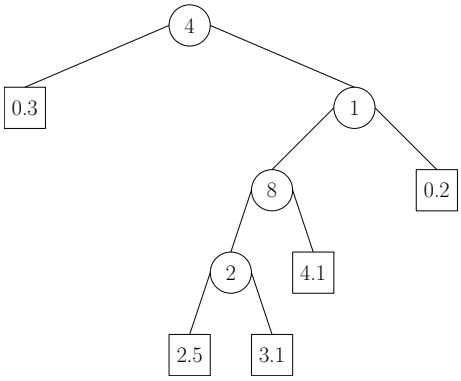
```
type ('a, 'b) arbre =
  | F of 'b
  | N of 'a * ('a, 'b) arbre * ('a, 'b) arbre
```

1 Fonctions élémentaires sur les arbres

1. Représenter graphiquement l'arbre t_1 suivant :

```
let t1 =
  N (12,
     N (4,
        N (7, F 20, F 30),
        N (14, F 1, F 2)),
     F 20)
```

2. Donner le type et la définition OCaml de l'arbre t_2 ci-dessous :



3. Écrire une fonction `hauteur: ('a, 'b) arbre -> int` qui calcule la hauteur d'un arbre, définie comme la longueur maximale d'un chemin reliant la racine à une feuille. On doit avoir `hauteur t1 = 3` et `hauteur t2 = 4`. Quelle est sa complexité ?
4. Écrire une fonction `taille: ('a, 'b) arbre -> int` qui renvoie le nombre total de nœuds (internes ou non) dans un arbre. On doit avoir `taille t1 = 9` et `taille t2 = 9`. Donner sa complexité .
5. Écrire une fonction `dernier: ('a, 'b) arbre -> 'b` qui renvoie l'étiquette de la feuille située le plus à droite de l'arbre. On doit avoir `dernier t1 = 20` et `dernier t2 = 0.2`. Quelle est sa complexité ?

2 Liste des étiquettes lors d'un parcours en profondeur

Version récursive classique

1. Donner la liste des étiquettes de l'arbre `t2` :
   (a) dans l'ordre préfixe.
   (b) dans l'ordre infixe.
   (c) dans l'ordre postfixe.
2. Écrire une fonction `affiche_prefixe: (int, int) arbre -> unit` qui affiche toutes les étiquettes des nœuds (internes ou non) de l'arbre passé en argument dans l'ordre préfixe. On utilisera :
   — `print_int: int -> unit` pour afficher un entier.
   — `print_newline: unit -> unit` pour revenir à la ligne.

```
utop # affiche_prefixe t1;;
12
4
7
20
30
14
1
2
20
- : unit = ()
```

3. Écrire de même des fonctions `affiche_infixe` et `affiche_postfixe`.
4. On souhaite écrire des fonctions permettant d'obtenir la liste des étiquettes d'un arbre dans un ordre spécifié (préfixe, infixe ou postfixe). Comme les étiquettes des nœuds internes et celles des feuilles peuvent être de types différents, il nous faut pour cela créer un type à deux variantes :

```
type ('a, 'b) token =
  | Ft of 'b
  | Nt of 'a
```

Les fonctions à écrire seront donc de type `('a, 'b) arbre -> ('a, 'b) token list`.

(a) Écrire une fonction `postfixe_naif` la plus simple possible, en vous permettant l'utilisation de `@`.

(b) Expliquer pourquoi cette fonction risque d'être inefficace.

(c) Écrire une fonction `postfixe` plus efficace, et préciser sa complexité. On utilisera une fonction

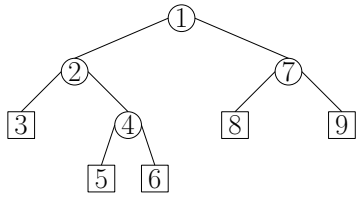
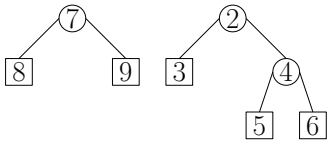
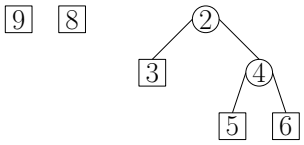
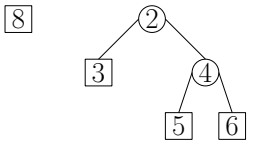
`aux: ('a, 'b) arbre -> ('a, 'b) token list -> ('a, 'b) token list`

telle que `aux t u` renvoie `v @ u`, où `v` est la liste des étiquettes de t dans l'ordre postfixe. *Cette fonction ne fera aucune concaténation : le* `v @ u` *est juste là pour donner la spécification.*

(d) En utilisant la même technique, écrire des fonctions `prefixe` et `infixe` efficaces.

Version récursive terminale

On souhaite écrire une version récursive terminale de la fonction `postfixe`. La difficulté vient du fait qu'un appel à `postfixe` sur un arbre de la forme `N (x, g, d)` donne *à priori* deux appels récursifs : un sur l'arbre `g` et un sur `d`. Évidemment, au plus l'un de ces appels peut être en position terminale. Pour surmonter cette difficulté, on va maintenir une liste d'arbres à traiter (on parle de *forêt*) ; à chaque étape, on récupèrera le premier élément de cette forêt, traitera sa racine, et, le cas échéant, ajoutera ses sous-arbres gauche et droit à la forêt. Le principe est illustré ci-dessous :

À traiter	Étiquettes lues
	
	1
	1 7
	1 7 9

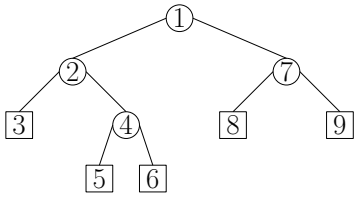
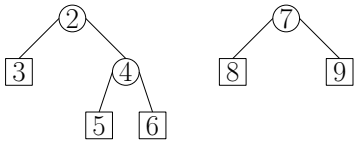
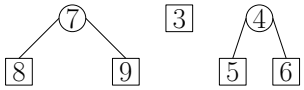
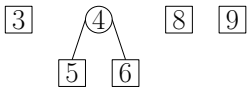
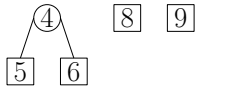
1. Poursuivre l'exécution simulée de l'algorithme jusqu'à sa conclusion.
2. Écrire une fonction `postfixe_term` ayant les mêmes spécifications que `postfixe` mais étant récursive terminale. On utilisera une fonction auxiliaire

   ```
   aux: ('a, 'b) arbre list -> ('a, 'b) token list -> ('a, 'b) token list
   ```

   prenant en entrée la forêt qui reste à traiter et les étiquettes déjà lues, et renvoyant la liste complète des étiquettes en suivant l'algorithme illustré ci-dessus.

3 Liste des étiquettes lors d'un parcours en largeur

On souhaite écrire une fonction `largeur: ('a, 'b) arbre -> ('a, 'b) token list` similaire à celle écrite dans la partie précédente. Pour ce faire, on va utiliser l'algorithme illustré ci-dessous :

À traiter	Étiquettes lues
	
	1
	1 2
	1 2 7
	1 2 7 3

1. Poursuivre l'exécution simulée de cet algorithme jusqu'à sa conclusion.
2. Expliquer pourquoi il est raisonnable d'utiliser une liste pour stocker les étiquettes lues, et pourquoi faire de même pour la forêt à traiter pose un problème de performances.
3. Écrire la fonction `largeur`. On pourra utiliser le module `Queue` de la bibliothèque standard.

```
Queue.create: unit -> 'a Queue.t
Queue.push : 'a -> 'a Queue.t -> unit
Queue.pop : 'a Queue.t -> 'a
Queue.is_empty : 'a Queue.t -> bool
```

Quelle est la complexité de cette fonction ?

4 Adressage

Dans un arbre binaire, un nœud situé à profondeur p (où la profondeur de la racine vaut 0) peut être repéré par une suite de p éléments $(x_0, \ldots, x_{p-1})$ de $\{\leftarrow, \rightarrow\}$: on part de la racine et pour $i = 0 \ldots p-1$ on descend vers le fils gauche ou droit du nœud actuel suivant que x_i vaut $\leftarrow$ ou $\rightarrow$. Informatiquement, les valeurs $\leftarrow$ et $\rightarrow$ seront codées par les booléens `false` et `true`.

Pour l'arbre `t1`, on obtient :

Adresse	Étiquette
ε	12
0	4
00	7
000	20
001	30
01	14
010	1
011	2
1	20

1. Dresser le tableau des adresses et étiquettes correspondantes pour l'arbre `t2`.
2. Écrire une fonction `lire_etiquette: bool list -> ('a, 'b) arbre -> ('a, 'b) token` prenant en arguments une adresse c et un arbre et renvoyant l'étiquette correspondante. Si l'adresse ne correspond à aucun nœud, on lèvera une exception à l'aide de `failwith`. Quelle est la complexité de cette fonction ?

```
utop # lire_etiquette [false; true; false] t1;;
- : (int, int) token = Ft 1
utop # lire_etiquette [false] t1;;
- : (int, int) token = Nt 4
```

3. (a) Écrire une fonction `incremente: (int, int) arbre -> bool list -> (int, int) arbre` prenant en entrée un arbre $\mathcal{T}$ et une adresse c et renvoyant l'arbre $\mathcal{T}'$ obtenu à partir de $\mathcal{T}$ en incrémentant de un l'étiquette du nœud d'adresse c. À nouveau, on lèvera une exception si l'adresse est invalide. Donner la complexité de cette fonction.
   (b) Dessiner `t1` et le nouvel arbre créé lors de l'appel de `incremente [false; true] t1` en faisant bien attention à ne pas dupliquer les noeuds qui sont partagés en mémoire par les deux arbres. Quelle est la complexité spatiale de `incremente` ?
4. Écrire une fonction `tableau_adresses: (int, int) arbre -> unit` affichant la liste des adresses de l'arbre avec les étiquettes correspondantes. On s'aidera de la fonction `affiche` suivante.

```
let affiche (x : int) (c : bool list) : unit =
  List.iter (fun b -> print_int (if b then 1 else 0)) c;
  Printf.printf " : %i\n" x
```

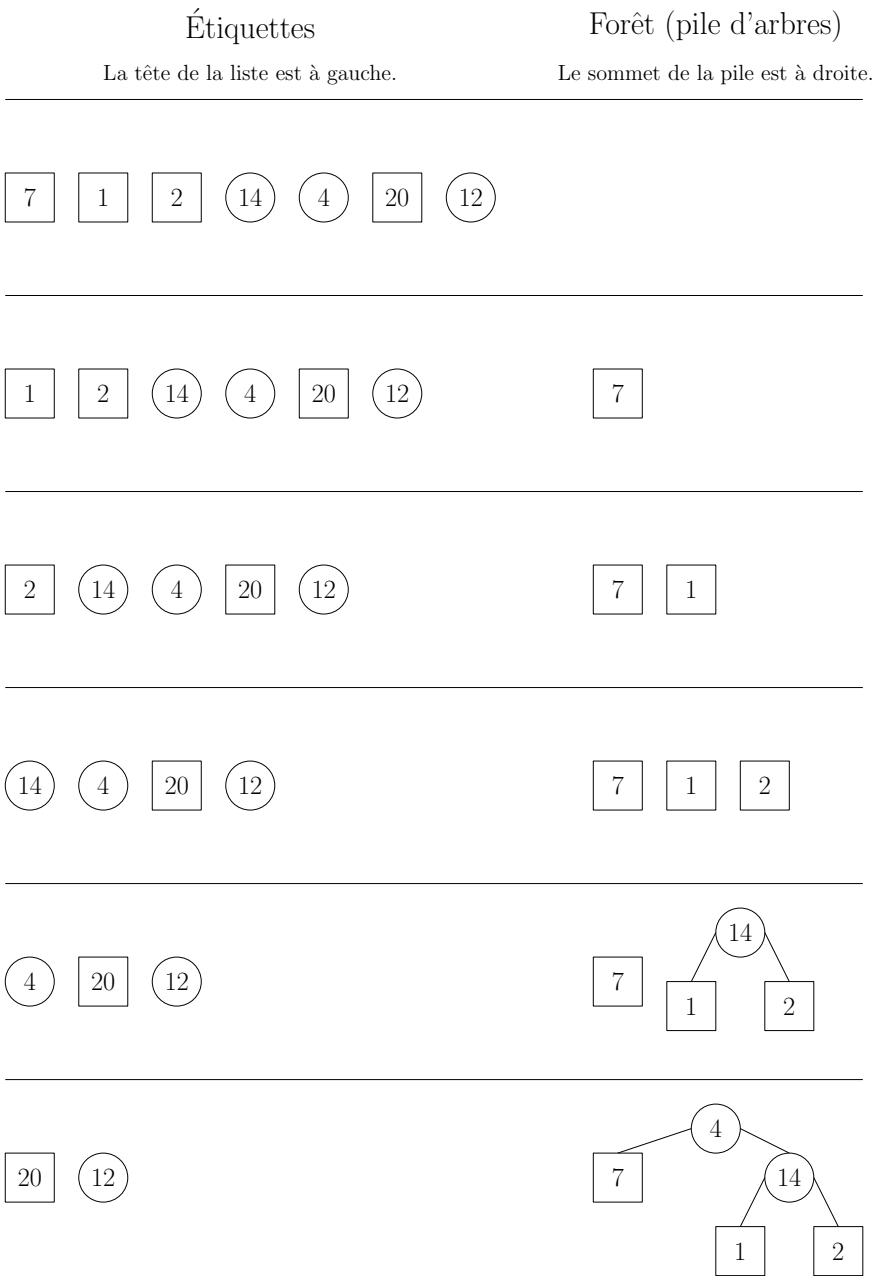
```
utop # affiche 12 [true; false; false];;
100 : 12
- : unit = ()
utop # tableau_adresses t1;;
 : 12
0 : 4
00 : 7
000: 20
001: 30
01 : 14
010 : 1
011 : 2
1 : 20
- : unit = ()
```

5 Reconstruction d'un arbre

On s'intéresse ici à la possibilité d'écrire des fonctions réciproques de `prefixe`, `postfixe`, `largeur`, `infixe`. Notre entrée est donc une `('a, 'b) token list` (dont on sait à quel ordre elle correspond), et notre sortie doit être l'unique `('a, 'b) arbre` donnant cette liste.

1. Donner deux arbres distincts dont le parcours infixe produit la liste `[Ft 0; Nt 1; Ft 2; Nt 3; Ft 4]`. Que peut-on en conclure ?

2. Pour reconstruire l'arbre à partir de son parcours postfixe, on propose l'algorithme illustré ci-dessous :



(a) Terminer la simulation de l'exécution de l'algorithme sur l'exemple donné.

(b) À l'aide d'une fonction auxiliaire

```
aux: ('a, 'b) token list -> ('a, 'b) arbre list -> ('a, 'b) arbre
```

écrire une fonction `lire_postfixe` reconstruisant un arbre à partir du résultat de son parcours postfixe.

3. Écrire une fonction `lire_prefixe`.

4. Écrire une fonction `lire_largeur` à l'aide d'une file.